**UNIVERSITI TEKNOLOGI MALAYSIA**

**FACULTY OF COMPUTING**

**CHAPTER 12 LAB MANUAL**

# Backend Security & Best Practices

*(Harden the Chapter 11 Books API)*

| FIELD | DETAILS |
|---|---|
| Course | SCSM2223 — Cross-Platform Application Development |
| Chapter | 12 — Backend Security and Best Practices |
| Lab time | 2 hours |
| Lab assessment | Lab exercise component |
| Software | Laragon, Composer, HeidiSQL, Sublime Text / VS Code |
| Builds on | Ch11_BooksAPI_Solution (copy as starting point) |
| | |

# 1. Lab Objectives

By the end of this lab you will be able to:

1. Add a reusable Validator class to enforce strict, whitelist-based input rules.
2. Encode JSON output safely so stored XSS payloads cannot escape into HTML.
3. Add a SecurityHeaders middleware (HSTS, CSP, X-Frame-Options, etc.).
4. Add a RateLimit middleware that returns 429 after too many failed logins.
5. Tighten the CORS policy to a known list of origins.
6. Add an ownership column on books and enforce IDOR protection on PUT.
7. Record security-relevant events to an audit_log table.

# 2. Prerequisites

Verify each item before starting. The fastest path is to copy your Chapter 11 project to a new folder named books-api-secure and work from there.

| # | Tool | Verify by running… | Expected |
|---|---|---|---|
| 1 | Laragon | Open Laragon → Start All | Apache + MySQL green |
| 2 | PHP 8.1+ | php -v | PHP 8.1.x |
| 3 | Composer | composer --version | Composer 2.x |
| 4 | Ch11 project | Login + protected calls work | JWT flow OK |

# 3. What We Will Build

Take the Chapter 11 secure-with-JWT API and add seven layers of hardening:

1. Schema upgrade — add books.created_by (FK to users.id) and an audit_log table.
2. Validator class — reusable, whitelist-based rules with friendly error messages.
3. JSON output encoding — JSON_HEX_TAG/JSON_HEX_AMP/etc. on every response.
4. SecurityHeaders middleware — HSTS, CSP, X-Frame-Options, etc. on every response.
5. RateLimit middleware — file-backed sliding-window limiter on /auth/login.
6. Cors middleware — allow-list driven by CORS_ALLOWED_ORIGINS env var.
7. IDOR protection — owner-or-admin check on PUT /api/books/{id}.

# Part A — Schema Upgrade

## Step 1 — Add created_by and audit_log

Run the following in HeidiSQL (or save it as sql/schema.sql and run with `mysql -u root < sql/schema.sql`):

```
USE books_api;
```

```
ALTER TABLE books ADD COLUMN created_by INT NULL AFTER genre,
  ADD CONSTRAINT fk_books_user FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET
NULL;

UPDATE books SET created_by = 1 WHERE id IN (1, 3);
UPDATE books SET created_by = 2 WHERE id = 2;

CREATE TABLE IF NOT EXISTS audit_log (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    occurred_at DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actor_id    INT         NULL,
    action      VARCHAR(50) NOT NULL,
    target      VARCHAR(80) NULL,
    ip_address  VARCHAR(45) NULL,
    detail      VARCHAR(500) NULL,
    INDEX idx_action (action),
    INDEX idx_actor  (actor_id)
) ENGINE=InnoDB;
```

| CHECKPOINT | DESCRIBE books; should show a created_by INT (NULL allowed). SELECT COUNT(*) FROM audit_log; should return 0. |
|---|---|

# Part B — Validator Helper

## Step 2 — src/Validation/Validator.php

Create the folder src/Validation/ and add this reusable class. Whitelist rules are clearer to read than ad-hoc if-statements.

```
<?php
namespace App\Validation;

final class Validator
{
    private array $rules = []; private array $required = [];
    public function field(string $n, callable $r, string $err): self {
        $this->rules[$n] = ['rule'=>$r, 'error'=>$err]; return $this;
    }
    public function required(string ...$n): self {
        $this->required = array_merge($this->required, $n); return $this;
    }
    public function validate(array $b, bool $partial = false): array {
        $errs = [];
        if (!$partial) foreach ($this->required as $n)
            if (!array_key_exists($n, $b)) $errs[$n] = "$n is required";
        foreach ($this->rules as $n => $r) {
            if (!array_key_exists($n, $b)) continue;
            if (!$r['rule']($b[$n])) $errs[$n] = $r['error'];
        }
        return $errs;
    }
    public static function nonEmptyString(int $max = 255): callable {
        return fn($v) => is_string($v) && trim($v) !== '' && mb_strlen($v) <= $max;
    }
```

```
    public static function intRange(int $a, int $b): callable {
        return fn($v) => is_numeric($v) && (int)$v >= $a && (int)$v <= $b;
    }
    public static function email(): callable {
        return fn($v) => is_string($v) && filter_var($v, FILTER_VALIDATE_EMAIL) !== false;
    }
}
```

### Step 3 — Use it in BookController::create()

```
$errors = (new Validator())
    ->required('title', 'author', 'year')
    ->field('title',  Validator::nonEmptyString(200), 'title must be 1-200 chars')
    ->field('author', Validator::nonEmptyString(150), 'author must be 1-150 chars')
    ->field('year',   Validator::intRange(1000, (int)date('Y')), 'year must be 1000..now')
    ->field('genre',  Validator::nonEmptyString(80),  'genre must be ≤ 80 chars')
    ->validate($body);

if ($errors) return $this->json($s, ['errors' => $errors], 400);
```

## Part C — XSS-safe JSON Output

### Step 4 — Update the json() helper in every controller

Add the JSON_HEX_* flags so any < > & ' " inside string values are escaped to \uXXXX in the JSON. Even if a frontend someday templates JSON straight into HTML, the payload cannot escape and execute.

```
private function json(Response $r, $data, int $status = 200): Response {
    $r->getBody()->write(json_encode(
        $data,
        JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE
        | JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT
    ));
    return $r->withHeader('Content-Type','application/json; charset=utf-8')
             ->withStatus($status);
}
```

**WHY** JSON.parse() in the browser still returns the original string with the angle brackets. The escape only matters when the JSON is embedded into HTML, which is exactly the path XSS exploits use.

## Part D — Security HTTP Headers

### Step 5 — src/Middleware/SecurityHeaders.php

```
<?php
namespace App\Middleware;
use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface;

final class SecurityHeaders implements MiddlewareInterface {
    public function process(ServerRequestInterface $req, RequestHandlerInterface $h):
```

```
ResponseInterface {
        return $h->handle($req)
            ->withHeader('X-Content-Type-Options',     'nosniff')
            ->withHeader('X-Frame-Options',            'DENY')
            ->withHeader('Referrer-Policy',            'no-referrer-when-downgrade')
            ->withHeader('Permissions-Policy',         'camera=(), microphone=()')
            ->withHeader('Strict-Transport-Security',  'max-age=63072000; includeSubDomains')
            ->withHeader('Content-Security-Policy',    "default-src 'self'; frame-ancestors
'none'");
    }
}
```

### Step 6 — Register it in public/index.php

```
$app->add(new App\Middleware\SecurityHeaders());  // ← added FIRST so it runs LAST
$app->add(new App\Middleware\JsonBodyParser());
$app->add(new App\Middleware\Cors());
```

## Part E — Rate Limiting on /auth/login

### Step 7 — src/Middleware/RateLimit.php (sliding window)

A simple file-backed limiter — perfect for the lab. In production you'd swap the store for Redis.

```
<?php
namespace App\Middleware;
use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface;
use Slim\Psr7\Response as SlimResponse;

final class RateLimit implements MiddlewareInterface {
    public function __construct(
        private int $limit,
        private int $window,
        private string $bucket = 'default'
    ) {}
    public function process(ServerRequestInterface $req, RequestHandlerInterface $h):
ResponseInterface {
        $ip   = (string)($req->getServerParams()['REMOTE_ADDR'] ?? 'unknown');
        $file = sys_get_temp_dir() . '/books-api-rate-' . preg_replace('/\W+/','_',$this-
>bucket) . '.json';
        $now  = time();
        $data = json_decode((string)@file_get_contents($file), true) ?: [];
        $b    = $data[$ip] ?? ['count'=>0, 'reset'=>$now + $this->window];
        if ($b['reset'] <= $now) $b = ['count'=>0, 'reset'=>$now + $this->window];
        $b['count']++;
        $data[$ip] = $b;
        @file_put_contents($file, json_encode($data), LOCK_EX);
        if ($b['count'] > $this->limit) {
            $r = new SlimResponse(429);
            $r->getBody()->write(json_encode(['error'=>'Too many requests']));
            return $r->withHeader('Content-Type','application/json')
                     ->withHeader('Retry-After', (string)max(1, $b['reset'] - $now));
```

```
        }
        return $h->handle($req);
    }
}
```

### Step 8 — Apply it to /auth/login (add in routes.php)

```
$loginMw = new RateLimit(
    (int)($_ENV['LOGIN_RATE_LIMIT']     ?? 5),
    (int)($_ENV['LOGIN_WINDOW_SECONDS'] ?? 60),
    'login'
);

$app->post('/auth/login', [$authCtrl, 'login'])->add($loginMw);
```

Add to .env:

```
LOGIN_RATE_LIMIT=5
LOGIN_WINDOW_SECONDS=60
```

## Part F — Tightened CORS

### Step 9 — src/Middleware/Cors.php (origin allow-list)

```
private array $allowed;

public function __construct() {
    $list = (string)($_ENV['CORS_ALLOWED_ORIGINS'] ?? '');
    $this->allowed = array_filter(array_map('trim', explode(',', $list)));
}

private function withCors($req, $res) {
    $origin = $req->getHeaderLine('Origin');
    $allow  = '*'; $creds = false;
    if ($this->allowed && in_array($origin, $this->allowed, true)) {
        $allow = $origin; $creds = true;
    }
    $res = $res
        ->withHeader('Access-Control-Allow-Origin',  $allow)
        ->withHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization')
        ->withHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS')
        ->withHeader('Vary', 'Origin');
    if ($creds) $res = $res->withHeader('Access-Control-Allow-Credentials', 'true');
    return $res;
}
```

Add to .env (comma-separated, no spaces):

```
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost:8080
```

## Part G — IDOR Protection on Update

### Step 10 — Update BookRepository::create() to record the owner

```
public function create(array $b, int $createdBy): int {
    $stmt = $this->pdo->prepare(
        'INSERT INTO books (title, author, year, genre, created_by)
         VALUES (:title, :author, :year, :genre, :owner)'
    );
    $stmt->execute([
        ':title' => trim($b['title']),    ':author' => trim($b['author']),
        ':year'  => (int)$b['year'],        ':genre'  => trim($b['genre'] ?? 'Uncategorised'),
        ':owner' => $createdBy,
    ]);
    return (int)$this->pdo->lastInsertId();
}
```

### Step 11 — Add the IDOR check inside BookController::update()

```
public function update(Request $r, Response $s, array $args): Response {
    $id   = (int)$args['id'];
    $book = $this->books->find($id);
    if (!$book) return $this->json($s, ['error'=>'Not found'], 404);

    $auth    = (array)$r->getAttribute('auth', []);
    $isOwner = (int)$book['created_by'] === (int)($auth['sub'] ?? 0);
    $isAdmin = ($auth['role'] ?? 'member') === 'admin';
    if (!$isOwner && !$isAdmin) return $this->json($s, ['error'=>'Forbidden'], 403);

    /* ... validate + update as before ... */
}
```

## Part H — Verify Each Defense

Restart the server: php -S localhost:8000 -t public, then run each test.

#### Test 1 — Security headers are present

```
GET http://localhost:8000/api/books

Inspect the response headers. Expect:
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  Strict-Transport-Security: max-age=63072000; includeSubDomains
  Content-Security-Policy: default-src 'self'; frame-ancestors 'none'
```

#### Test 2 — Rate limit on /auth/login

```
Run this 6 times back-to-back with the WRONG password:
  POST http://localhost:8000/auth/login
  { "email":"admin@books.test", "password":"wrong" }

1st-5th calls → 401 Invalid credentials
6th call      → 429 Too Many Requests + Retry-After header
```

#### Test 3 — Stricter validation

```
POST http://localhost:8000/api/books
Authorization: Bearer <token>
```

```
{ "title":"Future Book", "author":"X", "year": 9999 }

Expected: 400 with { "errors": { "year": "year must be 1000..now" } }
```

### Test 4 — XSS-safe output

```
POST http://localhost:8000/api/books
Authorization: Bearer <token>

{ "title":"<script>alert('hi')</script>", "author":"X", "year":2024 }

GET http://localhost:8000/api/books
→ The title is returned with < > escaped: "\u003Cscript\u003Ealert(...)\u003C/script\u003E"
```

### Test 5 — IDOR check

Login as member@books.test, then:

```
PUT http://localhost:8000/api/books/1   (book owned by admin)
Authorization: Bearer <member token>

{ "title":"Hijacked" }

Expected: 403 Forbidden

Switch to admin token, repeat → 200 OK
```

### Test 6 — CORS allow-list

```
OPTIONS http://localhost:8000/api/books
Origin: http://attacker.example

Expected: response has Access-Control-Allow-Origin: *  (NOT attacker.example)

OPTIONS http://localhost:8000/api/books
Origin: http://localhost:5173

Expected: Access-Control-Allow-Origin: http://localhost:5173
          Access-Control-Allow-Credentials: true
```

### Test 7 — Audit log captures events

After running tests 2 + 5, query the database:

```
SELECT occurred_at, action, target, ip_address FROM audit_log ORDER BY id DESC LIMIT 10;
```

**LESSON** Defense in depth: each test demonstrates ONE layer. If any one fails, the others still hold the line.

## Part I — Self-Check & Extension Tasks

1. Confirm that GET responses always include the six security headers.
2. Confirm that 6 wrong-password attempts in 60 seconds return 429.
3. Confirm that a member cannot update a book they do not own.
4. Confirm that an admin can update any book.

5. Confirm that POST /api/books with year > current year returns 400.
6. Confirm that audit_log contains rows for register, login.success, login.fail, book.create, book.update, book.delete.
7. BONUS — add per-account login limit (track failures by email, not just IP).
8. BONUS — add an /admin/audit endpoint that returns the audit_log (admin only).
9. BONUS — wire up Vue's Axios interceptor to attach the JWT and handle 429 with a back-off.

## Appendix — Common Errors & Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| Headers don't appear in the response | SecurityHeaders middleware not registered, or registered AFTER addRoutingMiddleware | Move $app->add(new SecurityHeaders()) BEFORE addRoutingMiddleware(). |
| Rate limit always returns 429 immediately | Stale rate-limit file with old buckets | Delete the JSON file under sys_get_temp_dir() (e.g. books-api-rate-login.json). |
| IDOR check sets created_by to NULL | Forgot to update create() to take $createdBy | Pass $auth['sub'] from the controller down to the repo. |
| CORS preflight 401 | AuthMiddleware running on OPTIONS | Add `$app->options('/{routes:.+}', ...)` and exclude OPTIONS from auth groups. |
| JWT_SECRET error on boot | Still set to placeholder | php -r "echo bin2hex(random_bytes(32));" → put in .env. |
| audit_log empty | Missing call to $audit->record(...) | Make sure the controller calls AuditLog at the right places. |

*End of Lab Manual.*